

# Tutorial de Programação em C

## Modularização, Ponteiros, Passagem de Parâmetros e Alocação Dinâmica

### Introdução

À medida que os programas crescem em tamanho e complexidade, torna-se necessário utilizar técnicas que facilitem sua organização, manutenção e reutilização. Entre essas técnicas destacam-se a modularização por meio de funções, o uso de ponteiros, a passagem de parâmetros por valor e por referência e a alocação dinâmica de memória.

Esses conceitos são fundamentais para o desenvolvimento de software e servem como base para tópicos mais avançados, como estruturas de dados, programação orientada a objetos e sistemas operacionais.

---

## 1. Modularização com Funções

### O que é modularização?

Modularizar um programa significa dividi-lo em partes menores chamadas funções.

Em vez de escrever todo o código dentro da função `main()`, podemos criar funções responsáveis por tarefas específicas.

### Vantagens

- Melhor organização do código;
  - Reutilização de funcionalidades;
  - Facilidade de manutenção;
  - Maior legibilidade.
- 

### Exemplo sem modularização

```
#include <stdio.h>

int main() {

    int a = 10;
    int b = 20;
    int soma = a + b;

    printf("Soma = %d\n", soma);
```

```
    return 0;
}
```

---

## Exemplo com modularização

```
#include <stdio.h>

int somar(int a, int b) {
    return a + b;
}

int main() {

    int resultado = somar(10, 20);

    printf("Soma = %d\n", resultado);

    return 0;
}
```

Neste exemplo a responsabilidade de realizar a soma foi transferida para a função `somar()`.

---

## Estrutura de uma função

```
tipo_retorno nome_funcao(parametros) {

    instrucoes;

    return valor;
}
```

Exemplo:

```
float calcularMedia(float n1, float n2) {

    return (n1 + n2) / 2;
}
```

## 2. Ponteiros

### O que é um ponteiro?

Um ponteiro é uma variável que armazena o endereço de memória de outra variável.

Exemplo:

```
int idade = 20;
```

Suponha que a variável esteja armazenada no endereço:

```
1000
```

Um ponteiro pode guardar esse endereço:

```
int *p;  
p = &idade;
```

---

### Operadores importantes

#### Operador &

Obtém o endereço de memória.

```
printf("%p", &idade);
```

---

#### Operador \*

Acessa o valor armazenado no endereço apontado.

```
printf("%d", *p);
```

---

### Exemplo completo

```
#include <stdio.h>  
  
int main() {
```

```
int numero = 50;

int *p = &numero;

printf("Valor: %d\n", numero);
printf("Endereco: %p\n", &numero);
printf("Valor pelo ponteiro: %d\n", *p);

return 0;
}
```

---

## 3. Passagem por Valor

Por padrão, a linguagem C realiza passagem de parâmetros por valor.

Isso significa que a função recebe uma cópia da variável original.

---

### Exemplo

```
#include <stdio.h>

void incrementar(int x) {
    x++;
}

int main() {

    int numero = 10;

    incrementar(numero);

    printf("%d\n", numero);

    return 0;
}
```

Saída:

```
10
```

O valor original não foi alterado.

---

## O que aconteceu?

Durante a chamada:

```
incrementar(numero);
```

é criada uma cópia:

```
numero = 10
```

```
x = 10
```

A função altera apenas a cópia.

---

## 4. Passagem por Referência

Para modificar a variável original utilizamos ponteiros.

---

### Exemplo

```
#include <stdio.h>

void incrementar(int *x) {
    (*x)++;
}

int main() {

    int numero = 10;

    incrementar(&numero);

    printf("%d\n", numero);

    return 0;
}
```

Saída:

```
11
```

Agora a função modifica diretamente a variável original.

---

## Visualização

Antes:

```
numero = 10
```

Endereço:

```
1000
```

A chamada:

```
incrementar(&numero);
```

envia:

```
1000
```

A função acessa o conteúdo desse endereço:

```
(*x)++;
```

e altera a variável original.

---

## 5. Troca de Valores Utilizando Ponteiros

Um exemplo clássico é a troca de valores entre duas variáveis.

```
#include <stdio.h>

void trocar(int *a, int *b) {
    int temp = *a;

    *a = *b;
    *b = temp;
}

int main() {
    int x = 10;
```

```
int y = 20;

trocar(&x, &y);

printf("%d %d\n", x, y);

return 0;
}
```

Saída:

```
20 10
```

---

## 6. Alocação Dinâmica de Memória

Até agora os vetores foram criados com tamanho fixo.

Exemplo:

```
int vetor[100];
```

Mas nem sempre sabemos quantos elementos serão necessários.

Para isso utilizamos alocação dinâmica.

---

## Biblioteca Necessária

```
#include <stdlib.h>
```

---

## Função malloc()

Reserva memória durante a execução do programa.

Sintaxe:

```
malloc(quantidade_de_bytes);
```

---

## Exemplo

```
int *vetor;  
  
vetor = malloc(5 * sizeof(int));
```

Memória reservada:

```
vetor[0]  
vetor[1]  
vetor[2]  
vetor[3]  
vetor[4]
```

---

## Utilização

```
for(int i = 0; i < 5; i++) {  
    vetor[i] = i * 10;  
}
```

---

## Liberando Memória

Após o uso:

```
free(vetor);
```

Toda memória obtida por `malloc()` deve ser liberada com `free()`.

---

## Função realloc()

Permite aumentar ou diminuir uma área já alocada.

Exemplo:

```
vetor = realloc(vetor, 10 * sizeof(int));
```

Agora o vetor possui espaço para 10 elementos.

## Exemplo Completo

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *vet;

    vet = malloc(3 * sizeof(int));

    vet[0] = 10;
    vet[1] = 20;
    vet[2] = 30;

    vet = realloc(vet, 5 * sizeof(int));

    vet[3] = 40;
    vet[4] = 50;

    for(int i = 0; i < 5; i++) {
        printf("%d ", vet[i]);
    }

    free(vet);

    return 0;
}
```

## Boas Práticas

1. Sempre verificar o retorno de `malloc()` e `realloc()`.

```
int *vet = malloc(10 * sizeof(int));

if(vet == NULL) {
    printf("Erro de alocao.\n");
}
```

1. Liberar toda memória alocada.

```
free(vet);
```

1. Não acessar posições além do tamanho alocado.
2. Inicializar ponteiros quando possível.

```
int *p = NULL;
```

## Resumo

| Conceito                | Objetivo                       |
|-------------------------|--------------------------------|
| Funções                 | Dividir o programa em módulos  |
| Ponteiros               | Armazenar endereços de memória |
| Passagem por valor      | Trabalhar com cópias           |
| Passagem por referência | Alterar variáveis originais    |
| malloc()                | Alocar memória dinamicamente   |
| realloc()               | Redimensionar memória          |
| free()                  | Liberar memória                |

## Exercícios

### Exercício 1 – Fácil

Crie uma função chamada `quadrado()` que receba um número inteiro e retorne o quadrado desse número.

A função deverá ser chamada pela `main()` e o resultado exibido na tela.

### Exercício 2 – Médio

Crie uma função chamada `trocar()` que receba dois números inteiros por referência e troque seus valores.

Exemplo:

Entrada:

10 20

Saída:

### Exercício 3 – Médio

Crie um programa que solicite ao usuário a quantidade de elementos de um vetor.

Utilize `malloc()` para criar o vetor dinamicamente, leia os valores e exiba a soma dos elementos.

### Exercício 4 – Difícil

Crie uma função chamada `maiorElemento()` que receba um vetor e seu tamanho e retorne o maior valor armazenado.

O vetor deve ser criado dinamicamente utilizando `malloc()`.

### Exercício 5 – Difícil

Crie um programa que permita ao usuário:

1. Informar inicialmente a quantidade de elementos;
2. Criar um vetor dinamicamente utilizando `malloc()`;
3. Ler os valores;
4. Aumentar o tamanho do vetor utilizando `realloc()`;
5. Ler os novos valores;
6. Exibir todos os elementos armazenados.

O programa deve liberar toda a memória utilizada antes de finalizar.